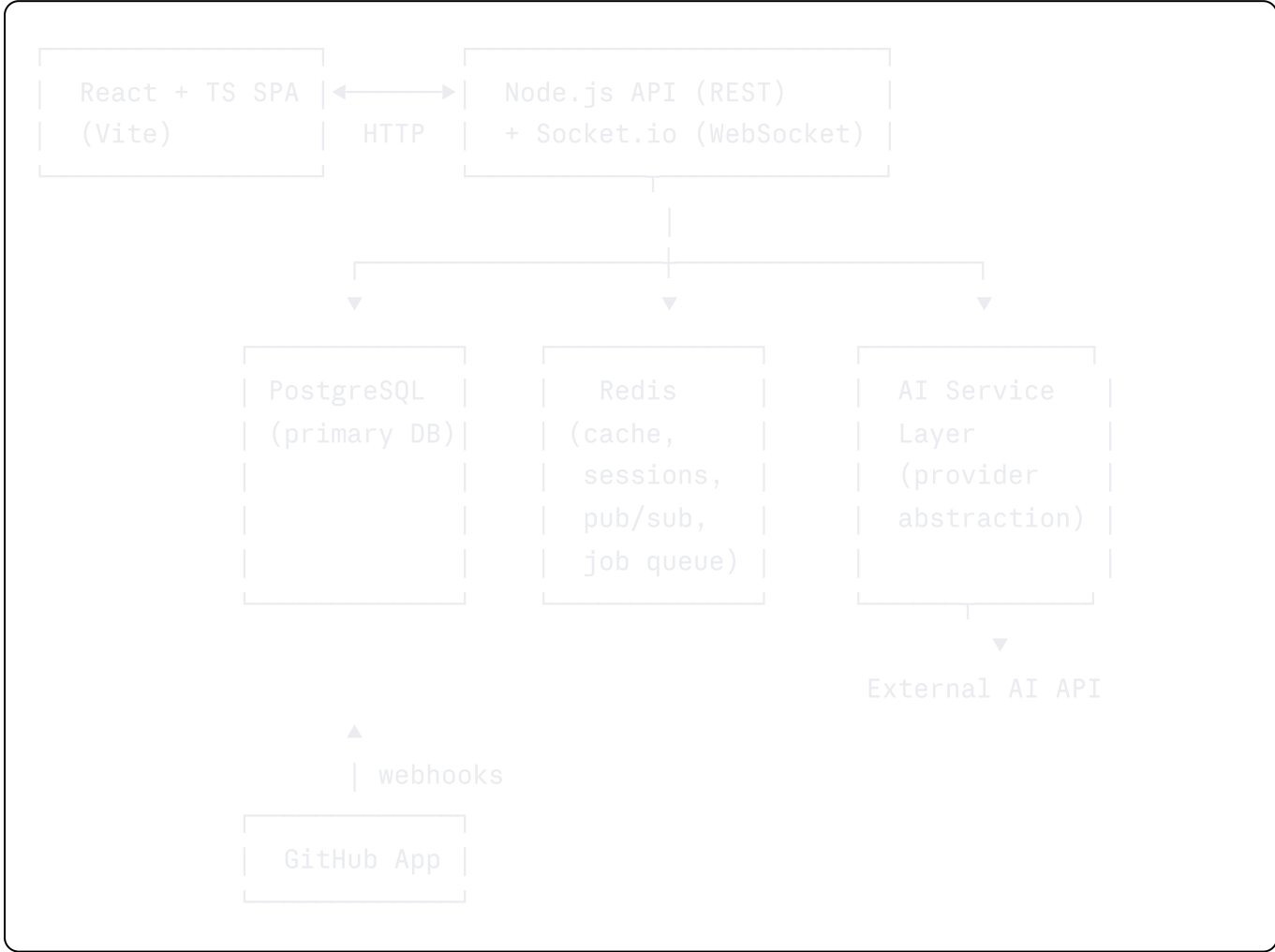


AI Software Development Workspace — Technical Development Plan

Companion to: `ai-dev-workspace-functional-documentation.md` **Stack:** React + TypeScript, Node.js, PostgreSQL, Redis, WebSockets, Docker, AI APIs, AWS

1. Architecture Overview



Single Node.js API service for MVP (not microservices) — REST endpoints, a Socket.io layer for real-time, and an internal AI Service module. Splitting into separate services is a later scaling decision, not a Phase 1 one.

2. Core Technical Decisions

Concern	Choice	Why
Backend framework	Express + TypeScript (Fastify as a drop-in alternative if perf matters later)	Lowest ceremony, you already know Node — avoid NestJS-level structure until team/complexity actually needs it
ORM	Prisma	Strong TS DX, migrations built in, easy to reason about for a solo/small-team build
Auth	JWT (access + refresh) , refresh tokens hashed in Redis	Stateless access token, revocable refresh token — standard, low-risk pattern
Real-time	Socket.io + Redis adapter	Handles reconnection/fallback out of the box; Redis adapter lets it scale past one instance later without a rewrite
Frontend build	Vite	Matches your existing Vitest preference, fast dev loop
Frontend server state	TanStack Query	Handles caching/refetching for REST data without hand-rolled hooks
Frontend local state	Zustand (only where needed — board UI state, modals)	Avoid a heavy global store for what's mostly server state
Rich text editor	Tiptap (ProseMirror-based)	Purpose-built for Notion-style block editing, good extension model
Drag-and-drop (Kanban)	@dnd-kit	<code>react-beautiful-dnd</code> is unmaintained; dnd-kit is the current standard
Background jobs	BullMQ (Redis-backed)	AI calls, webhook processing, and notifications shouldn't block request/response cycles
Testing	Vitest (frontend + backend)	Matches your existing preference
CI/CD	GitHub Actions	Free for this project size, integrates directly with the GitHub integration module

3. Repository Structure

```
/apps
  /web          # React + TS + Vite frontend
  /api          # Node.js + TS backend (REST + Socket.io)
/packages
  /shared-types # DTOs / interfaces shared between web and api
/infra
  docker-compose.yml
  Dockerfile.web
  Dockerfile.api
```

npm workspaces is enough for this size — no need for Nx/Turborepo unless build times become a real problem.

4. Database Schema — Phase 1

Prisma schema for the Phase 1 entities (Auth, Workspace, Project, Task, Sprint):

```
prisma
```

```
enum Role {
  ADMIN
  PM
  DEVELOPER
  REVIEWER
  VIEWER
}

enum TaskStatus {
  BACKLOG
  TODO
  IN_PROGRESS
  IN_REVIEW
  DONE
}

model User {
  id          String   @id @default(cuid())
  email       String   @unique
  passwordHash String?
  githubId    String?  @unique
  createdAt   DateTime @default(now())
  memberships WorkspaceMember[]
}

model Workspace {
  id          String   @id @default(cuid())
  name        String
  createdAt   DateTime @default(now())
  members     WorkspaceMember[]
  projects    Project[]
}

model WorkspaceMember {
  id          String   @id @default(cuid())
  userId      String
  workspaceId String
  role        Role
  user        User     @relation(fields: [userId], references: [id])
  workspace   Workspace @relation(fields: [workspaceId], references: [id])

  @@unique([userId, workspaceId])
}

model Project {
  id          String   @id @default(cuid())
```

```

workspaceId String
name         String
workspace    Workspace @relation(fields: [workspaceId], references: [id])
tasks        Task[]
sprints      Sprint[]
}

model Sprint {
  id          String    @id @default(cuid())
  projectId   String
  goal        String?
  startDate   DateTime
  endDate     DateTime
  project     Project   @relation(fields: [projectId], references: [id])
  tasks       Task[]
}

model Task {
  id          String    @id @default(cuid())
  projectId   String
  sprintId    String?
  title       String
  description  String?
  status      TaskStatus @default(BACKLOG)
  priority    String?
  assigneeId  String?
  dueDate     DateTime?
  storyPoints Int?
  project     Project   @relation(fields: [projectId], references: [id])
  sprint      Sprint?   @relation(fields: [sprintId], references: [id])
  createdAt   DateTime  @default(now())
}

```

Requirement, Doc, Snippet, Message, and GitHub-related models (PullRequest, AIReviewComment) get added in their respective phases below — no need to design them upfront.

5. API Design

REST, versioned under `/api/v1`. Every route (except auth) requires a valid access token and is scoped to a `workspaceId` resolved from the token — never trust a `workspaceId` passed in the request body for access control.

Phase 1 endpoints:

```
POST    /api/v1/auth/register
POST    /api/v1/auth/login
POST    /api/v1/auth/refresh
POST    /api/v1/workspaces
POST    /api/v1/workspaces/:id/invite
GET     /api/v1/workspaces/:id/projects
POST    /api/v1/projects
GET     /api/v1/projects/:id/board      # tasks grouped by status
POST    /api/v1/projects/:id/tasks
PATCH  /api/v1/tasks/:id              # status changes, edits
POST    /api/v1/projects/:id/sprints
PATCH  /api/v1/sprints/:id
```

6. Real-Time Architecture

- Socket.io server attached to the same Node process as the REST API (split out later only if load demands it).
 - Redis adapter (`@socket.io/redis-adapter`) from day one of Phase 3 — costs nothing extra now, avoids a rework when you eventually run more than one API instance.
 - Room convention: clients join `workspace:{workspaceId}` and `project:{projectId}` rooms on connect.
 - Core events: `task:moved`, `task:updated`, `presence:update`, `doc:cursor`, `chat:message`.
 - Every emitted event is scoped to a room — never broadcast workspace data globally.
-

7. AI Service Layer

One internal module (`/apps/api/src/services/ai/`) backs every AI feature — Requirement drafting, Task drafting, and Code Review — rather than each feature calling the AI API directly.

- **Provider abstraction:** `AIPProvider.complete(prompt: string): Promise<string>` — swapping or A/B testing models later touches one file, not every feature.
- **Prompt templates** live as separate versioned files per draft type (`prompts/task-draft.ts`, `prompts/requirement-draft.ts`, `prompts/code-review.ts`), not inlined in route handlers.
- **Response validation:** AI is asked to return JSON, and that JSON is validated against a **Zod schema** before it ever reaches the client. If validation fails, the field(s) in question are left blank rather than passing through malformed data (this is what makes FR-3.4 — "leave it blank rather than guess" — actually enforceable).
- **Reliability:** every AI call wrapped with a timeout (e.g. 10s) and a single retry; on failure, the API returns the plain empty form so the user isn't blocked.
- **Async where it doesn't need to be synchronous:** Code Review runs (triggered by GitHub webhook) go through a BullMQ queue, not the webhook request/response

cycle — GitHub's webhook delivery has its own timeout and shouldn't be coupled to AI latency.

8. GitHub Integration Architecture

- Use a **GitHub App**, not an OAuth App — finer-grained per-repo permissions and a webhook secret are built into the App model, which the earlier open question flagged as the better fit.
 - Webhook endpoint (`POST /api/v1/webhooks/github`) verifies the `X-Hub-Signature-256` header against the app's webhook secret before processing anything.
 - Installation access tokens are short-lived (~1hr) — fetch fresh per job rather than caching long-term.
 - PR events land in a BullMQ queue; a worker resolves the linked Task (via branch name convention or commit message task ID) and updates status.
-

9. Auth & Multi-Tenancy

- Access token: short-lived JWT (~15 min). Refresh token: longer-lived, stored hashed in Redis, revocable on logout (delete the Redis key).
 - Every tenant-scoped table carries a `workspaceId`. Enforce the filter in the **service/repository layer**, not just in controllers — a missed check in one route shouldn't leak another workspace's data.
 - Postgres Row-Level Security is worth knowing about but isn't a Phase 1 requirement — application-layer filtering is enough to start; revisit if the data-isolation bar rises later.
-

10. Local Development Setup

`docker-compose.yml` services: `postgres`, `redis`, `api`, `web`.

`.env.example` (`api`):

```
DATABASE_URL=postgresql://user:pass@localhost:5432/aidevworkspace
REDIS_URL=redis://localhost:6379
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
GITHUB_APP_ID=
GITHUB_APP_PRIVATE_KEY=
GITHUB_WEBHOOK_SECRET=
AI_API_KEY=
```

`docker-compose up` should be the only command needed to get Postgres + Redis running locally; `api` and `web` run via their own dev servers (`npm run dev`) pointed at those containers.

11. Development Phases — Technical Breakdown

Mirrors the 5-phase MVP plan from the functional documentation, with a concrete checkpoint per phase so each stage is verifiable before moving to the next.

Phase 1 — Core Loop

Build: Auth (register/login/refresh), Workspace + invite flow, Project CRUD, Task CRUD, basic Kanban board (fetch-on-load, no real-time yet), Sprint CRUD. **Checkpoint:** Two users can register, one creates a workspace and invites the other, both can create/move tasks on a board, and a hard refresh shows correct persisted state. No WebSockets, no AI yet — this just proves the data model and core loop.

Phase 2 — AI-Assisted Creation

Build: AI Service Layer (provider abstraction, prompt templates, Zod validation), Quick Create Task endpoint (FR-3.1-3.5), Requirement model + AI-Assisted Requirement Creation. **Checkpoint:** Typing the OTP-login example sentence produces a correctly pre-filled, editable Create Task form, and nothing is saved until the user clicks Create.

Phase 3 — Collaboration Surfaces

Build: Socket.io + Redis adapter, Docs (Tiptap) with page hierarchy, Code Snippets, Team Chat, presence system, real-time task-move broadcasting. **Checkpoint:** Two browser sessions logged in as different users see each other's task moves and chat messages instantly, and presence indicators update correctly.

Phase 4 — GitHub + AI Code Review

Build: GitHub App setup, webhook endpoint + signature verification, PR-to-Task linking, AI Code Review worker (via BullMQ, reusing the AI Service Layer from Phase 2). **Checkpoint:** Opening a PR against a connected repo, referencing a task ID in the branch name, links the PR to the task and posts an AI review comment within the queue's processing window.

Phase 5 — Analytics Dashboard

Build: Aggregation queries (cycle time, velocity, burndown), chart components (recharts), CSV export. **Checkpoint:** Dashboard reflects real data from a sprint that's been run through Phases 1-4, with numbers that match a manual count.

12. CI/CD & Deployment Plan

- **CI:** GitHub Actions — lint + typecheck + Vitest on every PR; build and push Docker images on merge to `main`.
- **Local/dev:** `docker-compose` (Section 10).
- **Early deployment (Phase 1-2):** a single small AWS setup — one EC2 instance (or App Runner) running the Docker Compose stack, RDS Postgres (smallest tier), ElastiCache Redis (smallest tier). This is enough to demo and get real usage without standing up ECS/Fargate prematurely.

- **Later scale-up (once it's more than a solo demo):** move `api`/`web` to ECS Fargate behind an ALB, keep RDS + ElastiCache, add S3 for attachments and CloudFront for the frontend static build. This is a config/infra change, not an application rewrite, if Section 1's architecture is followed from the start.
-

13. Getting Started — First Concrete Steps

1. Scaffold the monorepo (`/apps/web`, `/apps/api`, `/packages/shared-types`) with npm workspaces.
2. `docker-compose up` for Postgres + Redis; confirm both are reachable.
3. Set up Prisma with the Phase 1 schema (Section 4); run the first migration.
4. Build and manually test auth endpoints (register/login/refresh) with a REST client.
5. Build Workspace → Project → Task CRUD endpoints.
6. Build a minimal Kanban UI wired to the Task API — plain list view per column first, add `@dnd-kit` drag-and-drop only after the CRUD loop works.
7. **Verification checkpoint:** create a workspace, add a project, create and move a few tasks, refresh the browser, confirm everything persisted correctly. This is the Phase 1 exit criterion — don't start Phase 2 (AI) until this holds.